

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION SYSTEMS

Compiler Construction (CS F363)

II Semester 2025-26

Compiler Project

Coding Details

(March 02, 2026)

Group Number

50

1. Team Members Names and IDs

ID 2022B3A70635P	Name Samyak Savi
ID 2022B3A70637P	Name Chavi Gupta
ID 2022B5A70943P	Name Saumya Agarwal
ID 2022B3A70597P	Name Atharva Agrawal
ID 2022B3A70568P	Name Nishit Patel
ID 2022B3A71269P	Name Agrim Goyal

2. Mention the names of the Submitted files :

1. driver.c	7. parser.c	13. grammar.txt
2. lexer.c	8. parser.h	14. coding details.pdf
3. lexer.h	9. parserDef.h	15. t1-t6.txt
4. lexerDef.h	10. parser_helpers.c	16. testcase1-testcase5.txt
5. lexer_helpers.c	11. parser_helpers.h	17. Commentremoval_t2.txt
6. lexer_helpers.h	12. makefile	18.

3. Total number of submitted files: 26 (12 code files + 1 grammar file + 1 coding details.pdf + 12 testcases)

4. Have you compressed the folder as specified in the submission guidelines? yes

5. Lexer Details:

[A]. Technique used for pattern matching: DFA-based pattern matching using a hand-coded deterministic finite automaton.

Each state is a case in a switch-based loop in scan_next; processes one character at a time from twin buffer.

[B]. Keyword Handling Technique: Hash-map-based lookup (ReservedMap). After DFA recognizes an identifier lexeme.

[C]. Hash function: FNV-1a — start with offset basis 2166136261; for each character c: $h \wedge= c$; $h *= 16777619$; then take modulo table size 64.

Collisions handled via chaining with linked lists.

[D]. Have you used twin buffer? yes

[E]. Error handling and reporting: yes

[F]. Describe the errors handled by you: (1) Unknown symbols (e.g. \$, |); (2) Unknown patterns (malformed tokens: <-, <--, 23., &&, @, uppercase letters); (3) Variable identifiers exceeding 20 characters.

[G]. tokenInfo: TokData struct with kind (**enum TokKind**), text (**lexeme char array, max 128**), ln (**line number**), and nv (**NumVal union of int/double for numeric token values**).

6. Parser Details:

[A]. High Level Data Structure Description:

- i. grammar: Array of Production structs; each has head (non-terminal index), body array of Symbol structs (each with isTerm flag and index), and bodyLen. Loaded from grammar.txt at initialization.
- ii. FIRST and FOLLOW: 2D boolean arrays FIRST and FOLLOW are in FFsets as `fst[NUM_NONTERMINALS][NUM_TOKENS]`, `flw[NUM_NONTERMINALS][NUM_TOKENS]`, and `nullable[NUM_NONTERMINALS]`
- iii. parse table: 2D integer array `entry[NUM_NONTERMINALS][NUM_TOKENS]`; each cell stores a production index (≥ 0), `CELL_SYNC` (-3), or `CELL_EMPTY` (-1). Built from FIRST/FOLLOW sets automatically.
- iv. parse tree: Left-child right-sibling tree of PNode structs. Each node has: leaf flag, tok (TokData for leaves), nt (non-terminal for inner nodes), child (first child pointer), sib (next sibling pointer), and up (parent pointer). No dynamic children array.
- v. Any other: SymStack — linked list of SymFrame nodes via `SymFrame *prev` pointer; head points to the top of stack. DualBuf — twin-buffer for efficient file I/O with configurable block size and automatic refilling.

[B]. Parse tree

- i. Constructed: yes
- ii. Printing as per the given format: yes
- iii. Describe the order you have adopted for printing the parse tree nodes
Inorder traversal: implemented by `inorder_walk`. For each non-terminal node, visit the leftmost child first, then print the parent, then visit the remaining children left to right by following sib pointers. Leaf nodes printed when visited.

[C]. Grammar and Computation of First and Follow Sets

- i. Data structure: Array of Production structs with lhs, body (Symbol array with isTerm+index), bodyLen.
- ii. FIRST and FOLLOW sets computation automated: yes
- iii. Functions: Functions: `ff_compute` (computes FIRST and FOLLOW iteratively) in `parser.c`

[D]. Error Handling

- iv. Attempted: yes
- v. Errors handled: Lexical (unknown symbols/patterns, overlength IDs); syntax terminal mismatch (fabricated);
CELL_SYNC recovery (pop non-terminal); CELL_EMPTY panic mode (skip to anchor: ; , endif, endwhile, end, ; endrecord endunion endif endwhile else)] end main EOF).

7. Compilation Details:

- [A]. Makefile works: yes
- [B]. Code Compiles: yes
- [C]. Mention the .c files that do not compile: None — all files compile cleanly.
- [D]. Any specific function that does not compile: None
- [E]. Ensured the compatibility of your code with the specified gcc version: yes

8. Driver Details: Does it take care of the options specified earlier: yes

9. Execution

[A]. All test cases (t1-t6, testcase1-testcase5) execute correctly without crashes. Correct files produce parse tree; error files produce line-numbered reports. Parser recovers and continues parsing.

[B]. Gives segmentation fault with any of the test cases (1-6) uploaded on the course page. If yes, specify the testcase file name: No

10. Specify the language features your lexer or parser is not able to handle: None — all specified language features are handled.

11. Are you availing the lifeline: No

12. Declaration: We, Samyak Savi, Chavi Gupta, Saumya Agarwal, Atharva Agrawal, Nishit Patel, Agrim Goyal, declare that we have put our genuine efforts in creating the compiler project code and have submitted the code developed only by us. We have not copied any piece of code from any source. If our code is found plagiarized in any form or degree, we understand that a disciplinary action as per the institute rules will be taken against all of us in our team and we will accept the penalty as decided by the department of Computer Science and Information Systems, BITS, Pilani.

Samyak Savi 2022B3A70635P

Chavi Gupta 2022B3A70637P

Saumya Agarwal 2022B5A70943P

Atharva Agrawal 2022B3A70597P

Nishit Patel 2022B3A70568P

Agrim Goyal 2022B3A71269P

Date: March 02, 2026
